

Table of Contents

1.Abstract.....	1
2.Introduction.....	1
4.Scope.....	1
5.Prerequisites	2
6. Test Environment	2
7. Tools and Technologies	2
8. Application Under Test (AUT)	3
9. Test Scenarios	3
11. Test Cases:.....	4
12. Implementation	4
13. Test Execution Process.....	5
14. Test Reporting	5
15. Challenges.....	5
17. Best Practices.....	5
20. Sample Compatibility Matrix	6
21. Risks & Mitigation.....	6
22. Conclusion	6

Cross-Browser Testing Documentation Using Selenium WebDriver

1.Abstract

This project focuses on verifying that the Sauce Demo web application functions correctly across multiple web browsers including Chrome, Firefox and Chromium Edge. Cross-browser testing ensures compatibility, reliability and a consistent user experience. Selenium WebDriver with Python is used to automate testing and execute test cases efficiently.

2.Introduction

This document outlines the approach for ensuring that a web application functions correctly across multiple web browsers such as Chrome, Firefox and many more using Selenium WebDriver. Cross-browser testing helps identify compatibility issues and ensures a consistent user experience.

3.Objectives

- Validate application behavior across different browsers
- Ensure compatibility across Chrome, Firefox and Chromium Edge
- Detect UI/UX inconsistencies
- Ensure functionality works as expected in all supported environments
- Automate regression testing across browsers

4.Scope

Browsers Covered

- Google Chrome
- Mozilla Firefox
- Chromium Edge

Functional Areas

- Login Module
- Product Listing
- Cart Functionality
- Checkout Process
- Logout feature

4.1 In Scope:

- Cross-browser compatibility testing
- UI/UX consistency validation
- Functional testing across browsers
- Responsive design behavior
- JavaScript execution and rendering

4.2 Out of Scope:

- Security testing
- Load and stress testing

5. Prerequisites

5.1 Software Requirements

- Programming language (Python, JavaScript)
- Selenium WebDriver
- Browser drivers:
 - ChromeDriver (for Chrome)
 - GeckoDriver (for Firefox)
 - SafariDriver (build-in for Safari on macOS)
- Test framework (JUnit, Pytest)

5.2 Environment Setup

- Install browsers: Chrome, Firefox, Safari
- Set up environment variables for driver
- Ensure compatible driver versions with browsers

6. Test Environment

6.1 Browser:

- Google Chrome (latest, previous)
- Mozilla Firefox (latest, previous)
- Microsoft Edge (latest)
- Safari (Mac only)

6.2 Operating Systems:

- Windows 11

6.3 Tools:

- Selenium WebDriver
- pytest
- WebDriver Manager

7. Tools and Technologies

Category	Tool
Language	Python
Automation	Selenium WebDriver
Testing Framework	PyTest
IDE	PyCharm/VS Code
Reporting	PyTest HTML
Parallel Execution	PyTest-xdist

8. Application Under Test (AUT)

- Application Name: Sauce Demo
- URL: <https://www.saucedemo.com/>
- Type: E-commerce (Demo Application)
- Key Features:
 - User authentication (login/logout)
 - Product listing
 - Add to cart / remove from cart
 - Checkout process
 - Navigation menu

9. Test Scenarios

9.1 Login Functionality

- Valid login
- Invalid login
- Empty credentials

9.2 Product Page

- Verify product listing
- Add to cart
- Remove from cart

9.3 Cart Functionality

- View cart
- Update cart
- Proceed to checkout

9.4 Checkout Process

- Enter user details
- Complete purchase

9.5 Logout

- Successful logout

10. Testing Types Covered

10.1 Functional Testing

- Login functionality
- Product operations
- Cart and checkout flow

10.2 UI/UX Testing

- Layout consistency

- Font, color and alignment
- Button visibility and responsiveness

10.3 Compatibility Testing

- Browser rendering differences
- CSS and JavaScript compatibility

10.4 Responsive Testing

- Behavior on different screen sizes (desktop, tablet, mobile)

11. Test Cases:

Sauce Demo Test - Google Sheets

12. Implementation

12.1 Install Dependencies

Pip install selenium pytest pytest-xdist pytest-html

12.2 Driver Factory

From selenium import webdriver

Def get_driver(browser):

 if browser == "chrome":

 Return webdriver.Chrome()

 elif browser == "Firefox":

 return webdriver == "firefox":

 elif browser == "safari":

 return webdriver.Safari ()

 else:

 raise Exception ("Unsupported browser")

12.3 Sample Test Case

Import pytest

From selenium.webdriver.common.by import By

From utils.driver_factory import get_driver

@pytest.mark.parametrize("browser",["chrome","firefox"])

Def test_login(browser):

 driver = get_driver(browser)

```
driver.get(https://www.saucedemo.com/)  
driver.find_element(By.ID, "user-name").send_keys("standard_user")  
driver.find_element(By.ID, "password").send_keys("secret_sauce")  
driver.find_element(By.ID, "user-name").send_keys("standard_user")  
assert "inventory" in driver.current_url  
driver.quit()
```

13. Test Execution Process

- Launch target browser
- Navigate to Sauce Demo URL
- Execute predefined test cases
- Record results (pass/fail)
- Capture screenshots for defects
- Log issues in bug tracking tool

14. Test Reporting

Use PyTest HTML plugin:

Pip install pytest-html

Pytest -html=reports/report.html

15. Challenges

- Browser-specific rendering issues
- WebDriver compatibility with browser versions
- Handling dynamic elements
- Safari limitations on non-macOS systems

16. Common Cross-Browser Issues to Watch

- CSS misalignment (flex/grid issues)
- Font rendering differences
- Python compatibility errors
- Button click inconsistencies
- Responsive layout breaking on mobile

17. Best Practices

- Use explicit waits instead of sleep
- Keep tests independent

- Maintain reusable functions
- Regularly update Web Drivers

18. Entry & Exit Criteria

Entry Criteria:

- Application is accessible
- Test cases are prepared
- Test environment is ready

Exit Criteria:

- All critical test cases executed
- Major defects resolved
- Test summary report completed

19. Results & Outcome

- Verified consistent behavior across browsers
- Identified and fixed UI inconsistencies
- Improved application stability

20. Sample Compatibility Matrix

Feature	Chrome	Firefox	Edge	Status
Login	Pass	Pass	Pass	Pass
Add to cart	Pass	Pass	Pass	Pass
Checkout	Pass	Pass	Pass	Pass
Logout	Pass	Pass	Pass	Pass

21. Risks & Mitigation

Risk	Mitigation
Browser-specific bugs	Test early on all browsers
UI inconsistencies	Use responsive frameworks
Limited device access	Use cloud testing tools

22. Conclusion

Cross-browser testing using Selenium WebDriver in Python is an effective way to ensure that web applications perform reliably across environments. Automation reduces manual effort and increases testing efficiency.

Screenshots:

```
✓ 3 tests passed 3 tests total, 1 min 15 sec
testing started at 5:29 AM ...
Launching pytest with arguments C:\Users\DELL\PycharmProjects\SeleniumPytestFramework\tests\test_cart.py

===== test session starts =====
collecting ... collected 3 items

tests/test_cart.py::test_login[chrome]
tests/test_cart.py::test_login[firefox]
tests/test_cart.py::test_login[ChromiumEdge]

===== 3 passed in 75.23s (0:01:15) =====
PASSED [ 33%]Login successful!
Add to Card
Back to product
PASSED [ 66%]Login successful!
Add to Card
Back to product
PASSED [100%]Login successful!
Add to Card
Back to product

Process finished with exit code 0
```

report.html

Report generated on 14-Apr-2026 at 07:56:05 by pytest.html v4.2.0

Environment

Python	3.14.2
Platform	Windows-11-10.0.26200-SP0
Packages	• pytest: 9.0.3 • pluggy: 1.6.0
Plugins	• html: 4.2.0 • metadata: 3.1.1 • xdist: 3.8.0
JAVA_HOME	C:\Program Files\Java\latest\jdk-25

Summary

9 tests took 00:03:57.

(Un)check the boxes to filter the results.

☐ 0 Failed, ☒ 9 Passed, ☐ 0 Skipped, ☐ 0 Expected failures, ☐ 0 Unexpected passes, ☐ 0 Errors, ☐ 0 Reruns, ☐ 0 Retried,

Result	Test	Duration
Passed	tests/test_cart.py::test_login[chrome]	00:00:24
Passed	tests/test_cart.py::test_login[firefox]	00:00:32
Passed	tests/test_cart.py::test_login[ChromiumEdge]	00:00:27
Passed	tests/test_checkout.py::test_login[chrome]	00:00:36
Passed	tests/test_checkout.py::test_login[firefox]	00:00:43
Passed	tests/test_checkout.py::test_login[ChromiumEdge]	00:00:38
Passed	tests/test_login.py::test_login[chrome]	00:00:10
Passed	tests/test_login.py::test_login[firefox]	00:00:18
Passed	tests/test_login.py::test_login[ChromiumEdge]	00:00:09

```

✓ 3 tests passed 3 tests total, 1 min 53 sec

tests/test_checkout.py::test_login[chrome]
tests/test_checkout.py::test_login[firefox]
tests/test_checkout.py::test_login[ChromiumEdge]

===== 3 passed in 113.62s (0:01:53) =====
PASSED [ 33%]Login successful!
Add to Card
Back to product
Fill the form
Click Continue
Click Finish
PASSED [ 66%]Login successful!
Add to Card
Back to product
Fill the form
Click Continue
Click Finish
PASSED [100%]Login successful!
Add to Card
Back to product
Fill the form
Click Continue
Click Finish

Process finished with exit code 0

```

```

✓ 3 tests passed 3 tests total, 2 min 37 sec

===== test session starts =====
collecting ... collected 3 items

tests/Complete_test_workflow.py::test_workflow[chrome]
tests/Complete_test_workflow.py::test_workflow[firefox]
tests/Complete_test_workflow.py::test_workflow[ChromiumEdge]

===== 3 passed in 157.57s (0:02:37) =====
PASSED [ 33%]Login successful!
Add to Card
Add to Card 3
Remove to Card 3
Back to product
Fill the form
Click Continue
Click Finish
Back to Home
Logout page
PASSED [ 66%]Login successful!
Add to Card
Add to Card 3
Remove to Card 3

```